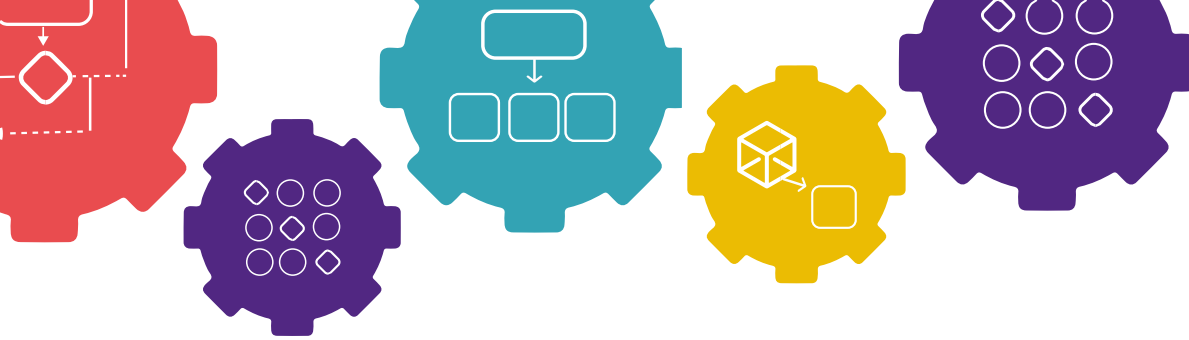




Laboratorio 15



Control de Calidad de las soluciones

Objetivos

- Anticipar y gestionar errores comunes durante el desarrollo de soluciones.
- Emplear técnicas básicas de depuración.



Conceptos clave

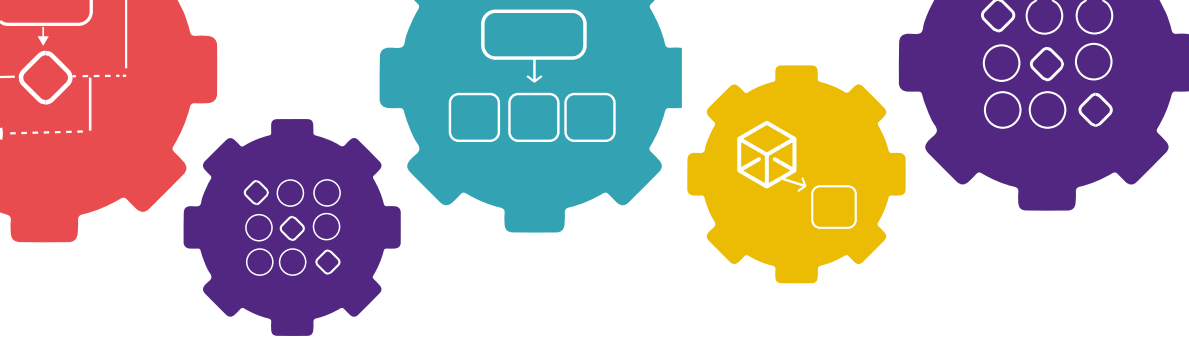
Defecto:

Es una falla en la solución que ocasiona un comportamiento equivocado. Existen varios tipos de defectos:

- De escritura
- Mala gramática y ambigüedad
- Inconsistencias
- Defectos lógicos y matemáticos.

Los defectos pueden ser detectados y eliminados en cualquier etapa del proceso de desarrollo, pero cuanto antes se encuentren será mejor. Algunos defectos aparecerán solo cuando la solución se ha implementado. En estos casos se requieren de pruebas (testing) y seguimiento paso a paso (debugging).

Pruebas de calidad: Verifican que el sistema desarrollado se comporta como se espera e identifica áreas específicas de fallo.

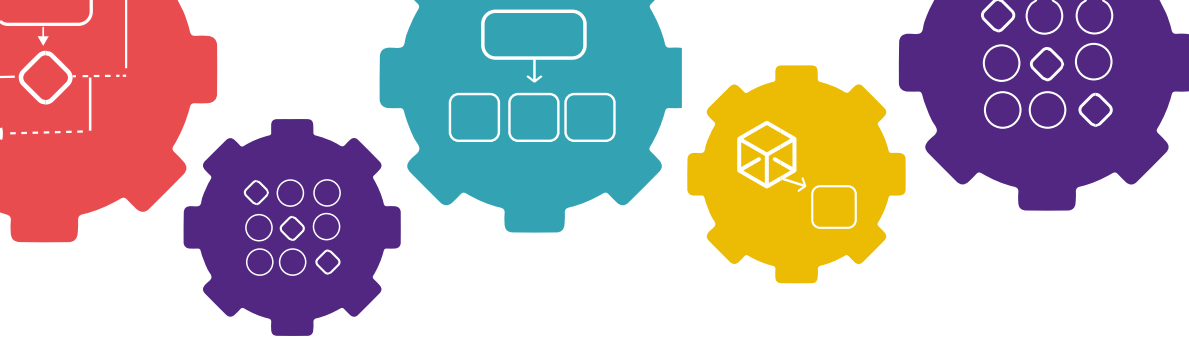


Seguimiento paso a paso: Enfoque sistemático que permite encontrar el error mientras el programa se ejecuta. Se emplea la descomposición para corregir comportamiento inesperados en tiempo real.



Ejercicios

1. **Tipos de Defectos**: Seleccione la opción que mejor describa el tipo de defecto presentado.
 - a. Un programa en C# para la recopilación de datos personales muestra el siguiente mensaje en pantalla:
"Ingresar su fecha de nacimiento y presionar Enter: "
Como resultado, el sistema ha sido incapaz de interpretar las respuestas de los usuarios debido a que todas vienen en formatos distintos (ej. DD/MM/YYYY, MM-DD-YYYY, YYYY-MM-DD, etc.)
 - Defecto de Ambigüedad
 - Defecto de Inconsistencia
 - Defecto Lógico
 - Defecto matemático
 - b. Se ha desarrollado un programa que calcula la suma, resta producto y cociente entre dos números ingresados por el usuario. Sin embargo, el programa presenta defectos cuando el usuario ingresa 0 en ambos valores pues no existe la división entre cero.
 - Defecto de Escritura
 - Defecto matemático
 - Defecto de Ambigüedad
 - Defecto de Inconsistencia.
 - c. Se solicita desarrollar un programa que genere 20 números pares aleatorios entre 50 y 100. Deberá mostrar el listado de números en pantalla, resaltando en color cian los números primos, y calcular las medidas de tendencia central.
 - Defecto de Ambigüedad
 - Defecto de Escritura
 - Defecto matemático
 - Defecto de Inconsistencia.



- d. Un sistema bancario en línea requiere que la contraseña de sus usuarios contenga entre 8 y 16 caracteres. A pesar de que se implementó una validación recientemente, se han detectado múltiples incidencias de fallo.

```
if (longitud >= 8 || longitud <= 16) {
    AceptarContraseña(contraseña)
} else{
    RechazarContraseña(contraseña)
}
```

- Defecto de Ambigüedad
- Defecto de Escritura
- Defecto lógico
- Defecto de Inconsistencia.

2. Pruebas de calidad (testing) – bottom up:

Elabore un plan de pruebas para validar el correcto funcionamiento de los siguientes métodos. Para cada método deberá realizar tres pruebas, indicando:

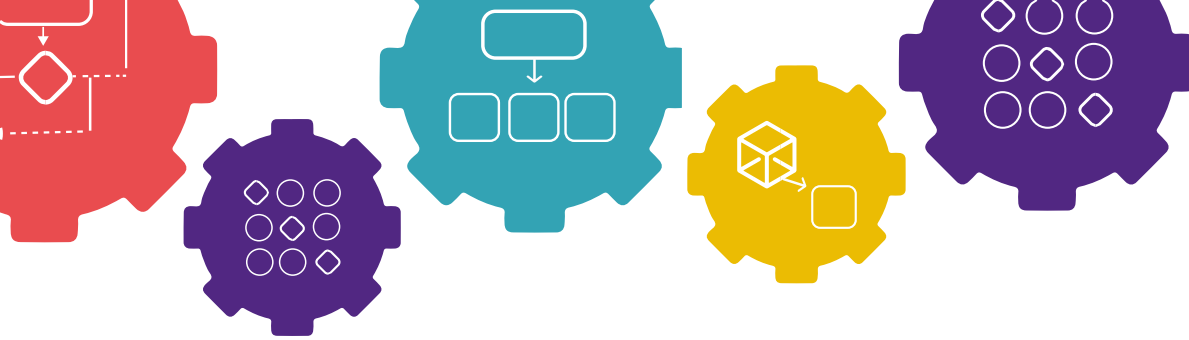
- Descripción de la prueba
- Valores de entrada
- Resultado Esperado
- Resultado obtenido, si el método cumple o no la prueba.

El objetivo del ejercicio es evidenciar, si los hay, los defectos en cada método y sugerir una propuesta de mejora.

Ejemplo: El método `CalcularPromedio()` recibe tres notas entre 0 y 100 y retorna el promedio de las tres.

```
0 referencias
static double CalcularPromedio(double nota1, double nota2, double nota3) {
    return (nota1 + nota2 + nota3) / 3;
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Notas dentro del rango	70, 80, 90	80	Aprobado
Nota Negativa	-10, 80, 90	Error o Rechazo	Reprobada
Nota mayor a 100	120, 80, 90	Error o Rechazo	Reprobada



Propuesta: Añadir una validación que determine si las entradas se encuentran en el rango de 0 a 100 antes de realizar los cálculos.

- a. El método `RetirarEfectivo()` permite retirar dinero de una cuenta bancaria por medio de un cajero automático. Esta recibe dos parámetros; El saldo disponible en la cuenta y el monto que desea retirar.

```
0 referencias
static void RetirarEfectivo(ref double saldo, double montoRetirado) {
    saldo = saldo - montoRetirado;
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Retiro válido	Saldo: 1000, Retiro: 300	Nuevo saldo: 700	Aprobado
Retiro mayor al saldo	Saldo: 500, Retiro: 700	Error o Rechazo	Reprobada
Retiro negativo	Saldo: 1000, Retiro: -100	Error o Rechazo	Reprobada
Retiro igual a cero	Saldo: 1000, Retiro: 0	Error o Rechazo	Reprobada

Propuesta de mejora

Validar que: El monto sea mayor que 0, el saldo sea suficiente y que no se permitan valores negativos o iguales a cero.

- b. La función `CalcularDescuento()` permite aplicar un descuento o rebaja al precio original de un producto. Para esto utiliza dos entradas; el precio original del producto y el porcentaje de descuento, entre 0 y 100, que debemos aplicar.

```
0 referencias
static double CalcularDescuento(double precio, double porcentaje)
{
    return precio - (precio * porcentaje / 100);
}
```

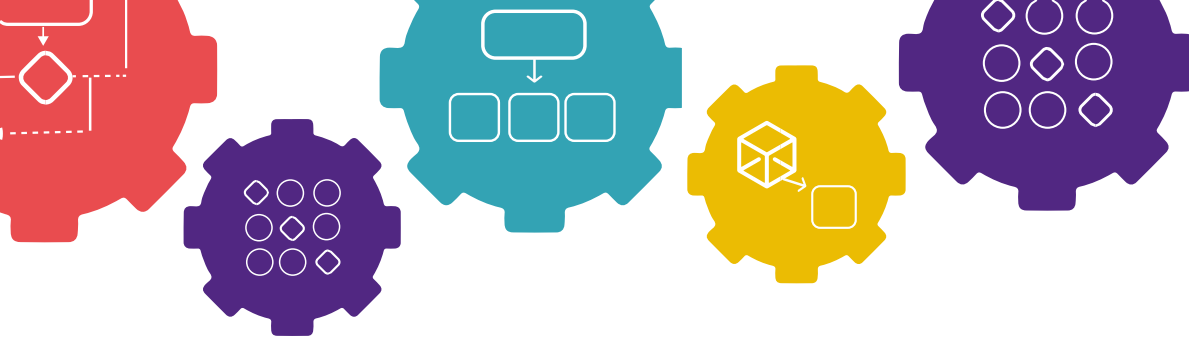
Caso	Entrada	Resultado Esperado	Resultado Obt.
Descuento válido	Precio: 200, Descuento: 10%	Precio final: 180	Aprobado
Porcentaje negativo	Precio: 200, Descuento: -20%	Error o rechazo	Reprobada
Descuento mayor a 100%	Precio: 200, Descuento: 150%	Error o rechazo	Reprobada
Precio negativo	Precio: -300, Descuento: 10%	Error o rechazo	Reprobada

Propuesta de mejora

Validar que: El porcentaje esté entre 0 y 100, el precio sea mayor que 0 y que no existan resultados negativos.

- c. El método `Depositar()` permite aumentar el saldo de una cuenta bancaria. Este recibe como entrada el saldo disponible del cliente y el monto que desea depositar.

```
0 referencias
static void Depositar(ref double saldo, double monto)
{
    saldo = saldo + monto;
}
```



Caso	Entrada	Resultado Esperado	Resultado Obt.
Depósito válido	aldo: 500, Depósito: 300	Nuevo saldo: 800	Aprobado
Depósito negativo	Saldo: 500, Depósito: -200	Error o rechazo	Reprobada
Depósito igual a cero	Saldo: 500, Depósito: 0	Error o rechazo	Reprobada
Depósito decimal válido	Saldo: 500, Depósito: 150.75	Nuevo saldo: 650.75	Aprobado

Propuesta de mejora

Validar que: El monto a depositar sea mayor que 0, no se permitan depósitos negativos o iguales a cero y se muestren mensajes de error cuando los datos sean inválidos.

3. Seguimiento paso a paso (Debugging):

Crear un proyecto de consola en C# e ingresar el código proporcionado para simular el pago de un crédito personal.

```
static void Main()
{
    double capital = 1000;
    double tasa = 0.05;
    double intereses = 0;
    double abonos = 0;

    for (int mes = 1; mes <= 12 && capital > 0; mes++)
    {
        // Cálculo de intereses del mes
        intereses = capital * tasa;

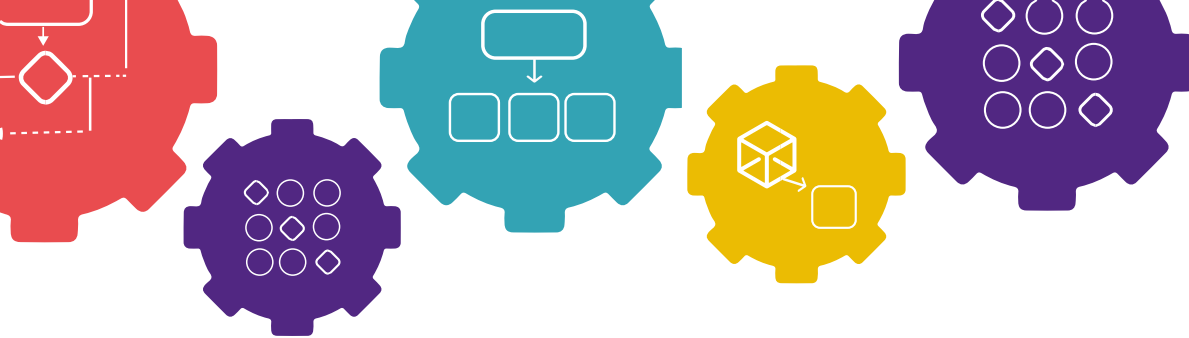
        // Abono realizado cada mes
        abonos = 100 + (mes * 10);

        // Actualización del capital
        capital = capital + intereses - abonos;
    }
}
```

Deberá utilizar herramientas de depuración (*debugging*) mediante el uso de **breakpoints** para observar cómo cambian las variables durante la ejecución del programa.

Instrucciones:

- Ejecute el programa en modo depuración (*Debug*).
- Coloque un **breakpoint** dentro del ciclo for, específicamente en la línea donde se actualiza el capital.
- Utilice las herramientas de depuración:
 - Step Over (F10)
 - Ventana de variables locales
- Observe cómo cambian las siguientes variables durante cada iteración:
 - capital
 - intereses
 - abonos



- tasa
 - mes
- e. Tome capturas de pantalla donde se visualicen claramente los valores de las variables al finalizar:

- Mes 2
- Mes 4
- Mes 6
- Mes 8

Al realizar la práctica en Rider para MacOS hubo problemas con el debugging, ya que el programa se ejecutaba y terminaba muy rápido, por lo que los breakpoints no se detenían correctamente dentro del ciclo for. Se intentó solucionar agregando `Console.ReadKey()` y `Console.ReadLine()`, además de revisar la configuración de los breakpoints, pero el problema continuó por la configuración del entorno en Rider. Aun así, se logró analizar el funcionamiento del código y conocer la utilidad del debugging.

Ejercicio	Criterio	Puntos	Total
Ejercicio 1: Tipos de Defectos	Inciso A	5 pts	20 pts
	Inciso B	5 pts	
	Inciso C	5 pts	
	Inciso D	5 pts	
Ejercicio 2: Testing	Diseño de pruebas adecuados, no faltan escenarios.	10 pts	40 pts
	Identifica correctamente todos los defectos presentes en las funciones	10 pts	
	Documenta de forma clara los resultados esperados y el comportamiento observado en cada prueba.	10 pts	
	Propone validaciones o correcciones adecuadas para solucionar los defectos encontrados	10 pts	
Ejercicio 3: Debugging	Estado de las variables en el Mes 2	10 pts	40 pts
	Estado de las variables en el Mes 4	10 pts	
	Estado de las variables en el Mes 6	10 pts	
	Estado de las variables en el Mes 8	10 pts	
Total			100 pts

